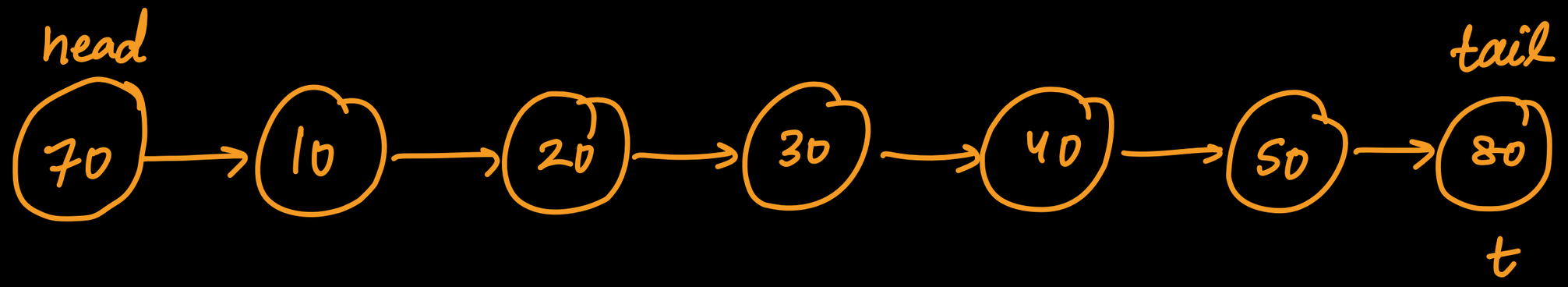
Linked List Class

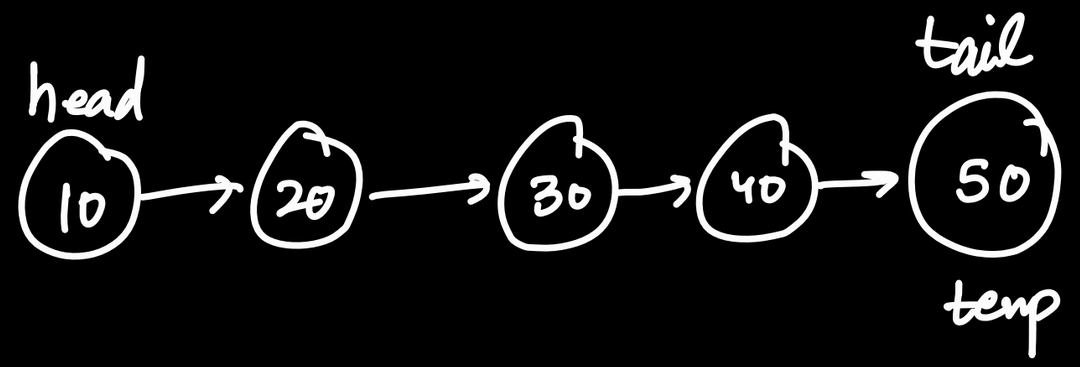



```
Node t = new Node(val);
tail.next = t;
tail = t;
```

insertAtTail()

T.C. = O(1)

A.S. = O(1)



create temp

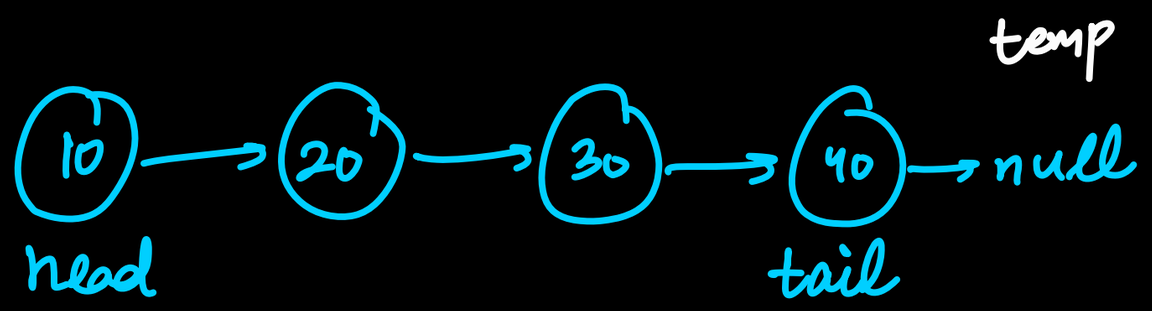
tail.next = temp

tail = temp

head = tail = temp;

display()

T.C. = O(n)

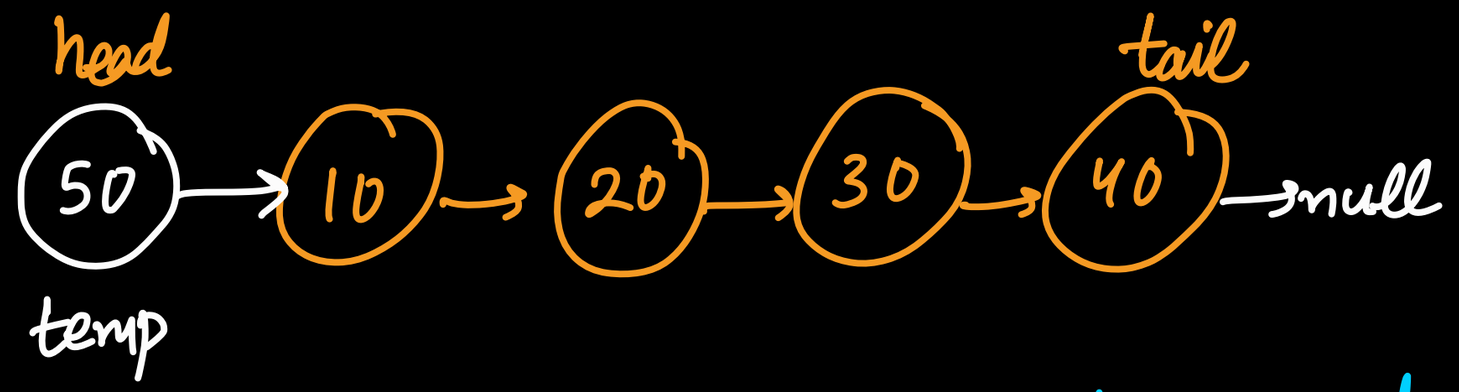



```
while (temp != null) {
    sout (temp.val);
    temp = temp.next;
}
```

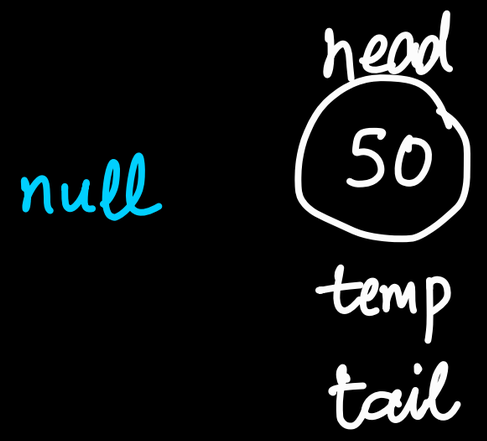
insertAtHead()

T.C. = O(1)

A.S. = O(1)



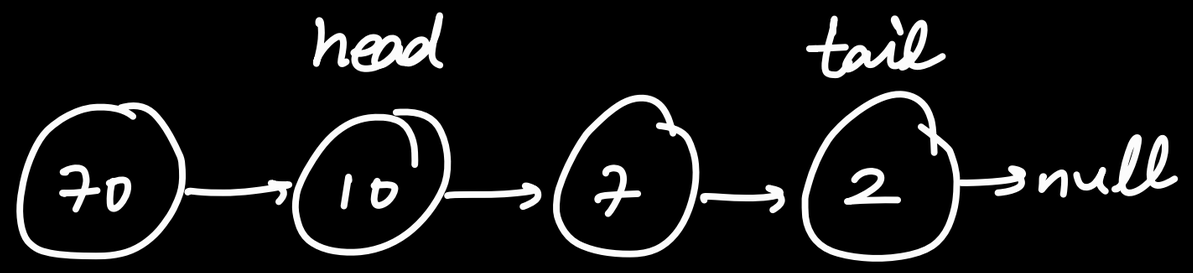

```
temp.next = head;
head = temp;
```



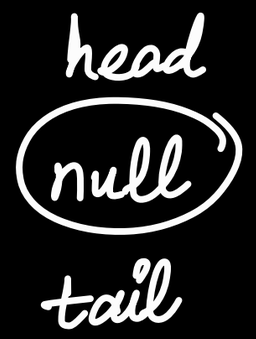
deleteAtHead()

T.C. = O(1)

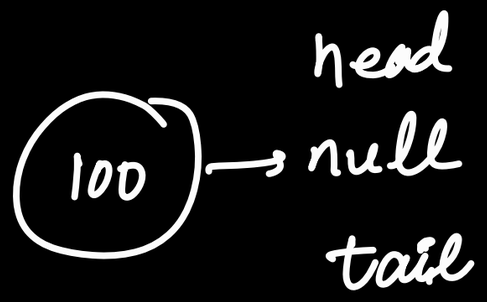
A.S. = O(1)



head = head.next



0 size



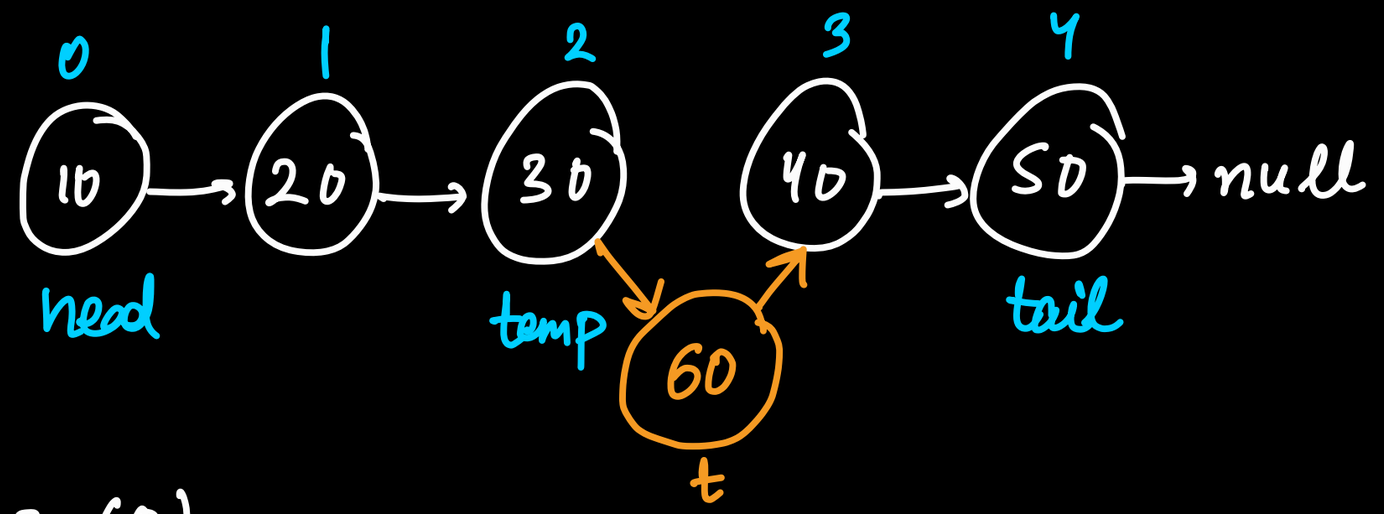
length()

O(1)

insert(idx, val)

T.C. = O(n)

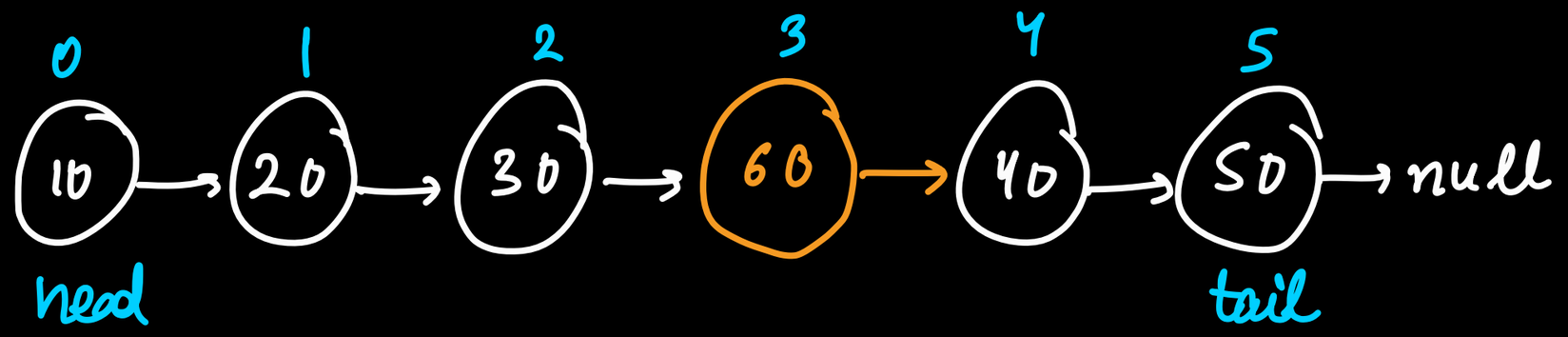
A.S. = O(1)



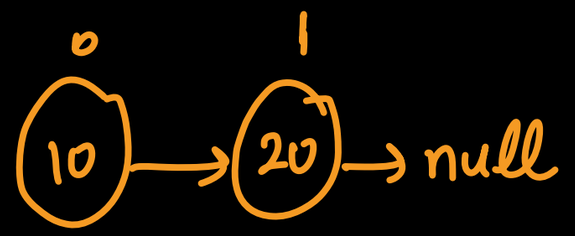
t.next = temp.next

temp.next = t

```
ll.insert(3, 60);
```



insert(idx)



size=2

insert(80, size) → addAtTail(80)

insert(70, 0) → addAtHead(70)

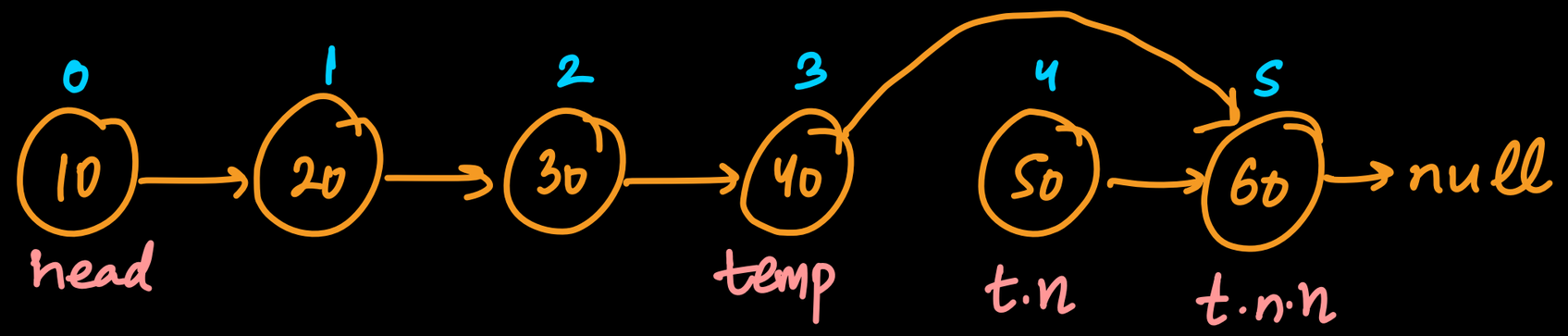
if(idx > size or idx < 0) error msg

get(idx)

O(n) Time

delete(idx) → breaking & attaching

T.C. = O(n)
A.S. = O(1)

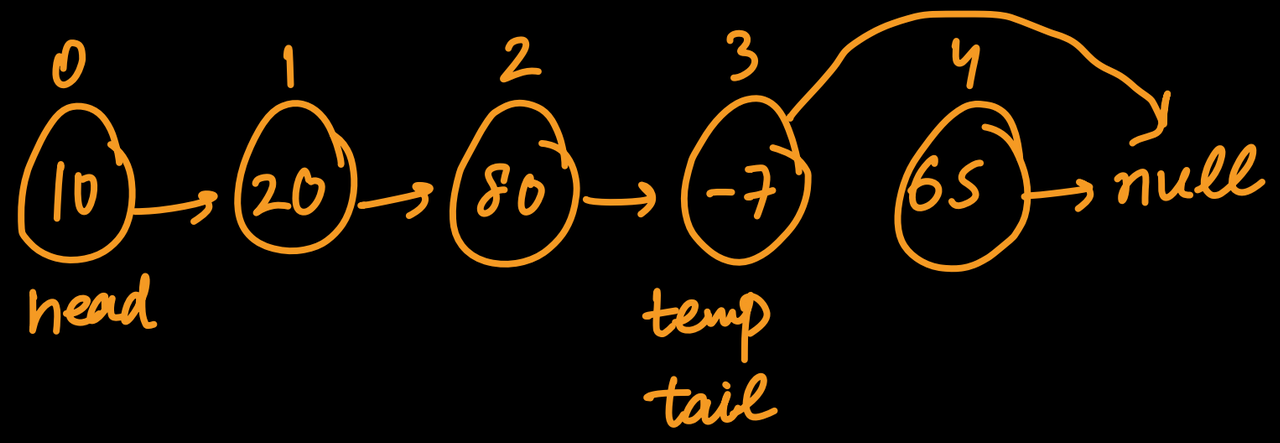



```
ll.delete(4);

temp.next = temp.next.next;
```

delete(idx)

at tail if(idx == size-1)

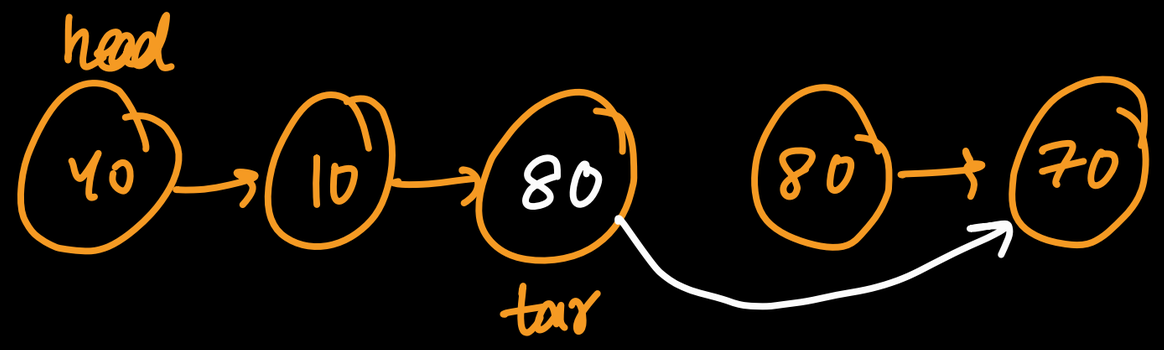


delete(4)

Linked-List ki Limitations

get is O(n) time

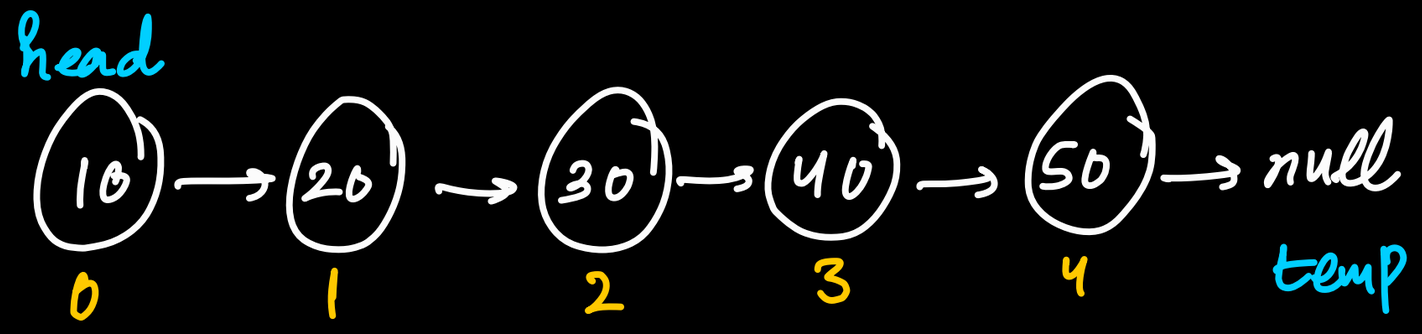
Ques: Delete in a singly Linked List



tar.val = tar.next.val

tar.next = tar.next.next

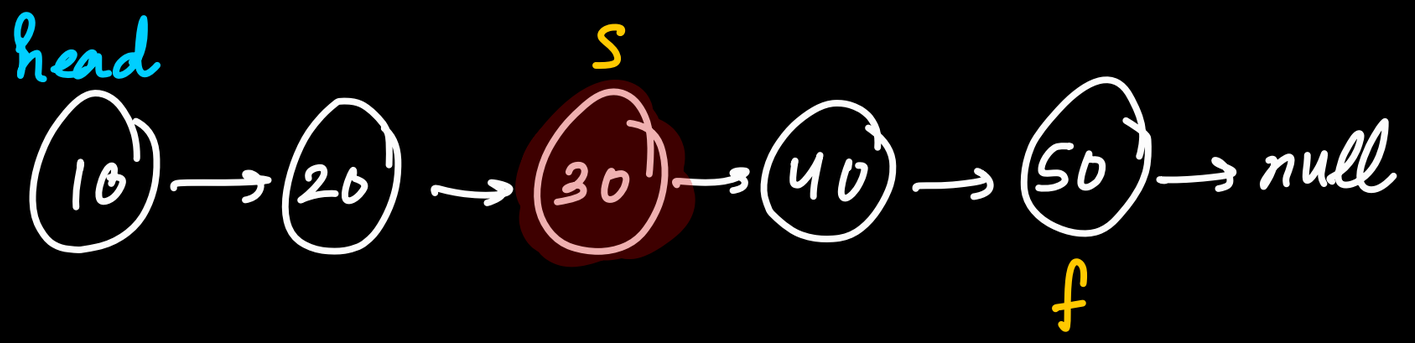
Ques: Middle of a Linked List



length = 0 1 2 3 4 5

Ques: Middle of a Linked List (Slow Fast Approach)

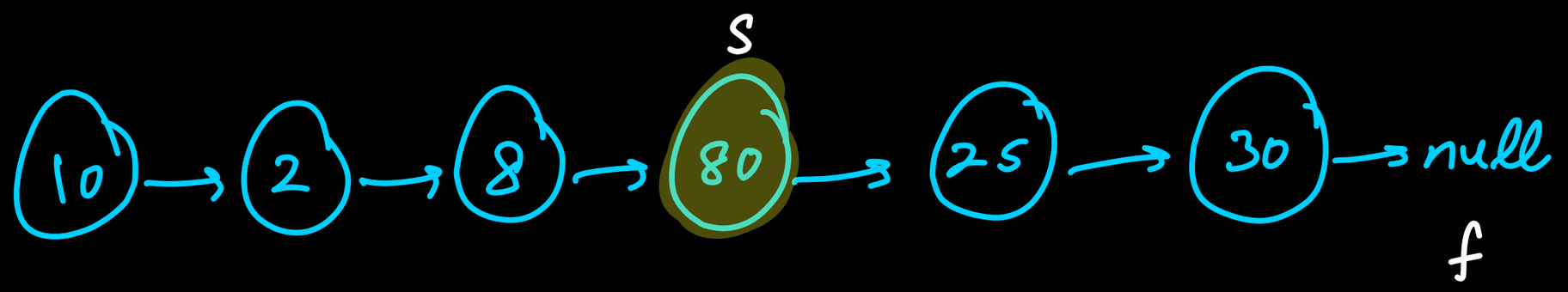
Odd Length




```
while (fast.next != null) {
    s = s.next
    f = f.next.next
}
return s.data;
```

Ques: Middle of a Linked List

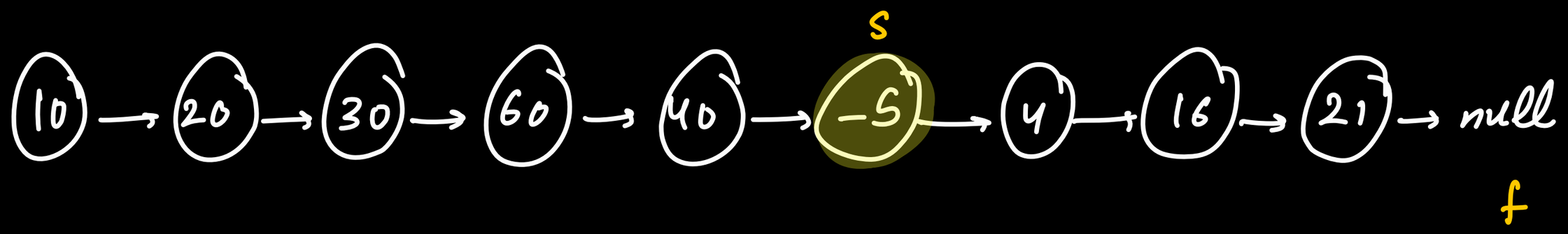
Even Length




```
while ( fast != null ) {
    s = s.next
    f = f.next.next
}
return s.data;
```

HW: Delete Middle of Linked List

Ques: Kth from end of Linked List



K=4 ans = -5

len = 9

n^{th} from end is actually $(len - n + 1)^{th}$ from start

~~Ques~~: Remove nth node from end of List

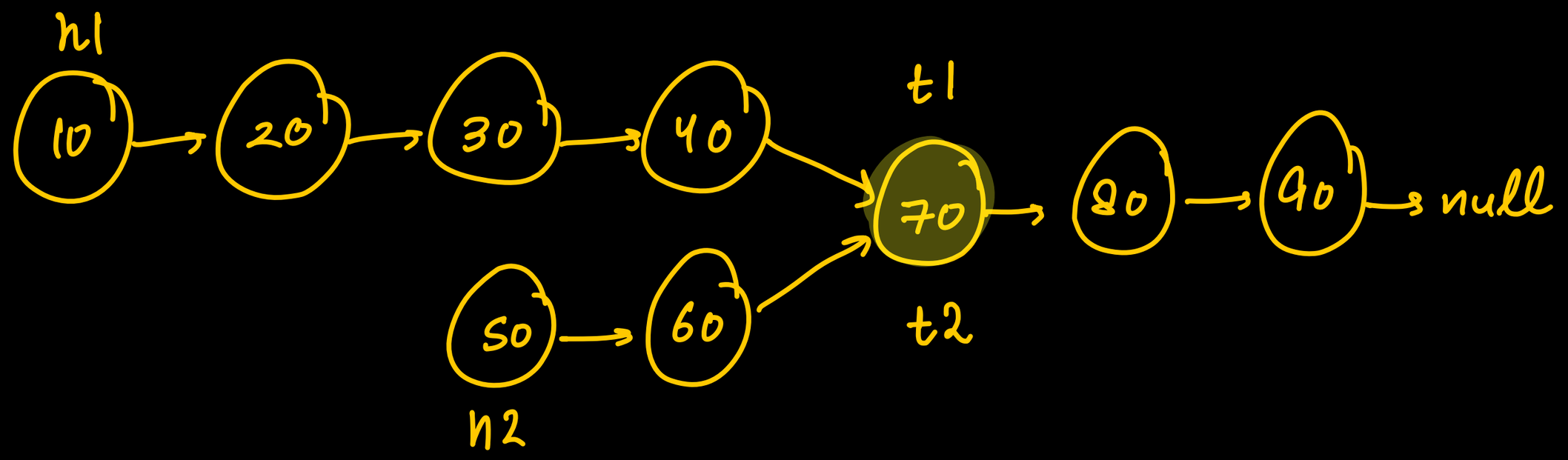
HW

Leetcode 19

HW: Swapping nodes in a Linked List

Leetcode 1721

Ques: Intersection of Two Linked Lists



Leetcode 160